

See-See-U

114-2 Introduction to AI, InnoFest Research Report

Group 10 (InnoFest group number 120)

114zu1001 王紫君

114zu1009 黎芯好

114zu1015 汪宜家

114zu1025 趙芯悅

114zu1058 楊恆慈

Project Description

Background

In the era of social media, people frequently share captivating photos of their daily lives, study spaces, or travels. However, social media users often encounter several pain points when trying to identify the locations in these photos:

- **Hidden Locations:** Influencers often gatekeep locations, requiring followers to direct message them just to get a place's name.
- **Unfamiliar Recommendations:** When seeing a recommendation from a follower they are not personally close with, users may feel awkward asking for the location directly.
- **Locating Friends:** Students often wonder where their peers are studying on campus to join them, but tracking down the exact campus building or library can be inefficient

Our survey of 41 samples revealed that **65.9% of people** have experienced the desire to know a specific location after seeing a photo shared by others. Among various categories, libraries showed a significantly high frequency of interest, with **63.4% of respondents** stating that a search scope limited to National Chengchi University (NCCU) libraries would be highly helpful.

Therefore, to address this practical need, our team decided to kickstart this project by focusing on the campus libraries and learning centers of National Chengchi University as our primary scope and pilot study.

Importance and Value

Our project, "**SEE-SEE-U**", addresses these gaps by offering an intuitive, image-based localization assistant. The system provides several key advantages:

- **Direct Recognition:** It recognizes specific locations directly from an image without requiring prior textual tags or location metadata.
- **Seamless Navigation:** It bridges the gap between image recognition and real-world utility by immediately offering navigation support, making it highly user-friendly.

- **Efficiency:** By automating the workflow, it eliminates the need to manually message others or guess locations, allowing users to discover places and connect with the world effortlessly.

Dataset Description

Since there was no comprehensively pre-existing, public image dataset specifically for National Chengchi University landmarks, our team **built the dataset from scratch through self-collection.**

The dataset was gathered through two primary methods to ensure diversity in lighting, angles, and composition:

- **Web Scraping:** Collecting publicly available images of NCCU libraries from various social media platforms, search engines, and the official university website.
- **Field Photography:** Team members personally visited the campus to take high-quality, real-world photos of the target buildings under different weather conditions and perspectives.

Dataset Scale and Structure

- **Classes:** The dataset targets **8 distinct categories** of campus libraries and major learning centers at NCCU.
- **Volume:** To ensure an un-biased and balanced dataset, we collected exactly **100 image samples per library category.**
- **Total Size:** The final dataset comprises a total of **800 labeled images**, which were split into training, validation, and testing sets for our CNN model.

Analytical Methods Applied

The system employs a **Convolutional Neural Network (CNN) model** for core feature extraction and image classification. The workflow consists of image pre-processing, feature extraction via the CNN model, and location mapping. Finally, the highest-probability prediction is converted into a Google Maps search keyword to dynamically generate a one-click navigation link.

Getting Started

Prerequisites and Environment

The project is explicitly implemented and optimized to run on the **Google Colab** environment, leveraging Google Drive for dataset cloud storage and model serialization.

- **Operating Platform:** Google Colab (Linux-based cloud instance).
- **Hardware Acceleration:** GPU acceleration is strongly required due to the size of the ResNet50V2 architecture and the two-phase training process.
- **Programming Language:** Python 3.9+
- **Deep Learning Framework:** TensorFlow 2.x (Keras API)

Dependencies and Package Installation

The system relies on several computer vision and data processing libraries.

Key Python Libraries Used:

- *tensorflow>=2.10.0* — Core framework harnessing ResNet50V2 with Adam optimizers and ImageDataGenerator to train the classification layer.
- *numpy* — Used for processing image matrices and multi-dimensional array operations.
- *IPython* — Used to render interactive HTML navigation links in Colab.

To download images for the dataset, users need to install the *bing-image-downloader* utility.

Data Setup and Preparation

- **Google Drive Integration:** The training script automatically mounts user's Google Drive to access the dataset.
- **Directory Path:** Ensure the image dataset is structured into 8 separate folders, one for each library, and placed under the following path in your Google Drive
- **Data Splitting:** The pipeline utilizes Keras's *ImageDataGenerator* to normalize pixel values (*rescale=1./255*) and automatically partitions the dataset into **80% Training data and 20% Validation data**.

- **Image Specifications:** All input images are automatically reshaped to a target size of **224* 224 pixels** with RGB channels.

Running the System

Step 1: Training - Transfer Learning

The system instantiates a ResNet50V2 backbone pre-trained on ImageNet weights, initially freezing all feature-extraction layers.

Custom top classification layers are appended: *MaxPooling2D()*, a flatten layer, a dense layer with a dropout regularizer, and a final 8-node *Softmax* layer.

The model compiles using the standard *Adam* optimizer and *Categorical Crossentropy* loss, training for **10 epochs** to warm up the classification head.

Step 2: Training - Fine-Tuning

To maximize accuracy, the script unfreezes the base model but restricts updates by freezing all layers except for the top 30 layers.

The model is re-compiled using a highly conservative learning rate to carefully tune deep feature maps without warping pre-trained capabilities, running for an additional **10 epochs**.

Upon completion, the model structure and final weights are saved directly to Google Drive as model.v2.keras.

Step 3: Interactive Prediction & Safety Filter

When executing the prediction cells, upload an evaluation image from the local file system. The model processes the image and generates a prediction matrix:

1. **Preliminary Safety Check:** If the highest prediction probability falls below a conservative threshold of 0.40, the code halts execution and triggers a safety print: *"Prediction: Unable to identify this image. Please upload another photo."*
2. **Final Mapping and Hyperlink Verification:** If the evaluation passes the initial block, the code maps the top predicted category to a localized scheme containing predefined coordinates. If the final output confidence scores above 0.75, the system prints out the final location along with a clickable interactive

HTML link. Clicking this link will immediately forward the user into Google Maps via their browser to guide them seamlessly to their chosen study destination.

File Structure

Below is the directory blueprint of our "SEE-SEE-U" project repository, outlining the exact organization of our final dataset, model weights, and execution scripts:

- [AI 導論 final pics/](#) (Dataset Directory)

Contains our self-collected campus image data. It includes exactly 800 image samples balanced evenly across 8 NCCU library and center categories:

- NCCU Main Library
- NCCU Dah Hsian Library
- NCCU Social Sciences Library
- NCCU Commerce Library
- NCCU Law Library
- NCCU Research Center and Innovation Incubation Center
- NCCU College of Communication Library
- NCCU Art Culture Center

- [models/](#) (Model Weights Directory)

Stores our deployment-ready deep learning model weights file named [model.v2.h5](#). This file contains the optimized mathematical parameters of our fine-tuned [ResNet50V2](#) network, which utilizes Transfer Learning to leverage deep feature knowledge from pre-trained image databases.

- [src/](#) (Core Source Code Directory)

Contains the modular Python pipeline scripts that run our application:

- [data_preprocessing.py](#): Manages data ingestion pipelines via Keras [ImageDataGenerator](#) with a 0.2 validation split, automatically reshaping image tensors to 224* 224* 3 and executing target

normalization.

- *train_model.py*: Constructs the sequential top layers over the frozen ResNet50V2 base, compiles the architecture using the *Adam* optimizer alongside *categorical_crossentropy* loss, and executes the 10-epoch training performance loop.
- *app.py*: Houses the interactive deployment and predictive inference engine. It feeds uploaded images into our saved model network, computes the highest target index score, filters predictions using a strict classification threshold, and automatically triggers embedded Google Maps navigation links.
- *requirements.txt* (Environment Configuration)
A standard reference file specifying mandatory module dependencies and library framework versions (*TensorFlow*, *Keras*, *NumPy*, *IPython Display*, etc.) required to safely reproduce the project environment.
- *README.md* (Repository Documentation)
The main markdown-formatted user manual that displays our comprehensive project overview, data distributions, training diagnostics, and deployment setup guidelines.

File Relationship Summary

Image samples organized within *AI 導論 final pics/* are structurally prepared and preprocessed by *src/data_preprocessing.py*. These formatted arrays are passed into *src/train_model.py* to train the top classification layers attached to the **ResNet50V2 Transfer Learning core**. Once the training cycle successfully concludes, the resulting optimal neural weights are saved into *models/model.v2.h5*.

Finally, our live deployment framework loads this file. When a user submits a campus photo, the system evaluates the top classification probability. If this score meets or exceeds our **0.75 confidence threshold**, it displays the confirmed location and provides a clickable Google Maps navigation hyperlink. If the prediction score falls below 0.75, it flags the sample as unidentifiable and prompts the user to upload another photo.

Analysis

Analysis Methods

- **Image Pre-processing & Data Pipeline:** We utilized a dataset of 800 self-collected images across eight NCCU library categories. To prepare this data for the model, we implemented a pipeline using the *Keras ImageDataGenerator*. Images were rescaled ($1./255$) and resized to a target size of **224* 224 pixels**. A **validation split of 0.2** was applied to monitor the model's ability to generalize to unseen campus photos.
- **Transfer Learning Architecture:** Our analysis leveraged the *ResNet50V2* model. We utilized transfer learning by freezing the base layers and adding a custom Sequential top layer, allowing the model to adapt its pre-trained "deep feature knowledge" to the specific architectural nuances of NCCU libraries.
- **Operational 8-Step Workflow:** The technical analysis follows a rigorous 8-step logic depicted in our demonstration:

Step 01: Image Input (X)

The process begins when a user uploads a photo of an NCCU library. This serves as the raw input (X) for our analytical pipeline.

Step 02: Pre-processing

The system immediately executes data preparation using a Keras-based pipeline. This includes resizing the image to 224* 224 pixels and normalizing pixel values (rescaling by $1./255$) to ensure compatibility with the deep learning model's input requirements.

Step 03: Feature Extraction

We utilize a CNN model to analyze the visual components of the photo. The model extracts deep architectural features, such as the unique structural patterns of the Dah Hsian Library or the Research Center.

Step 04: Clarity Evaluation

The system evaluates the extracted features to assess image clarity. This step is crucial for ensuring that the visual data is distinct enough to produce a reliable classification.

Step 05: Decision Analysis

The model performs a categorical prediction. For example, it can identify the NCCU Research Center with a **high confidence score of 0.9972**. To maintain reliability, we apply a 0.75 confidence threshold; results below this bar are flagged for re-uploading to prevent incorrect navigation.

Step 06: Location Mapping

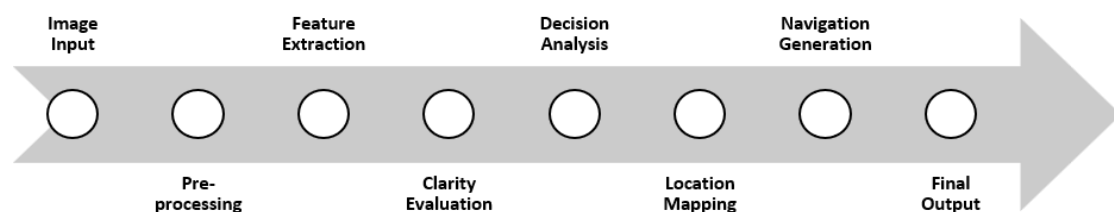
Once a location is identified, the system automatically converts the library name into standardized Google Maps search keywords. This bridges the gap between image recognition and geographic data.

Step 07: Navigation Generation

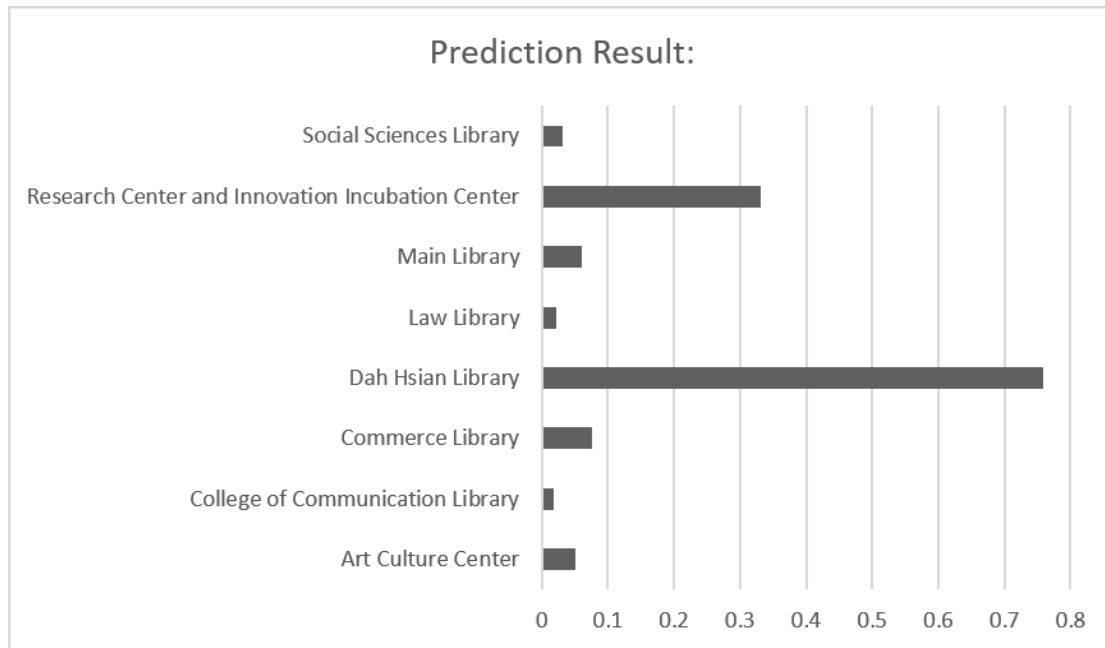
The system generates an automatic Google Maps navigation link. This process eliminates the need for the user to manually type or search for the building name.

Step 08: Final Output (Y)

The final results (Y) are displayed to the user, showing the predicted location name and providing a link to open Google Maps with one click for immediate navigation support.



The graphic below displays **the model's decision-making logic using probability scores**. In this example, the system identifies the Dah Hsian Library with a **confidence score of 0.7594**, effectively distinguishing it from other libraries with lower probabilities. By implementing a 0.75 confidence threshold, we ensure that the system only provides navigation for highly certain results, maintaining a "user-friendly" and reliable experience.



Survey Data and Theme Alignment

The core theme of "SEE-SEE-U" addresses a real-world social media issue: users see attractive photos but cannot find their locations. Our survey shows **85% of respondents** have wanted to visit a place in a post but could not identify the location, and **72% gave up searching** due to the social awkwardness of asking unfamiliar creators. These statistics prove that hidden location information is a widespread daily frustration. Additionally, the survey revealed that even when the location is known, switching apps to copy addresses and open maps is highly tedious. This insight guided our design, highlighting the need for a single tool that simplifies these friction points into direct navigation.

Technical Strategy Alignment

Operating within a mobile interface requires a meticulous balance between processing efficiency and inference accuracy. To prevent latency from heavy deep learning models and ensure a seamless user experience, we selected a lightweight CNN architecture designed for near-instant landmark recognition. Furthermore, anticipating that mobile users might frequently upload low-quality, blurry, or accidental images, we implemented a 0.75 confidence threshold as an automated gatekeeper. This design ensures the system only triggers a Google Maps navigation link when highly confident, guaranteeing both rapid processing speeds and reliable campus navigation.

Comparison

Custom-Trained CNN vs. Using Transfer Learning

Figure 1: Training and Validation Accuracy

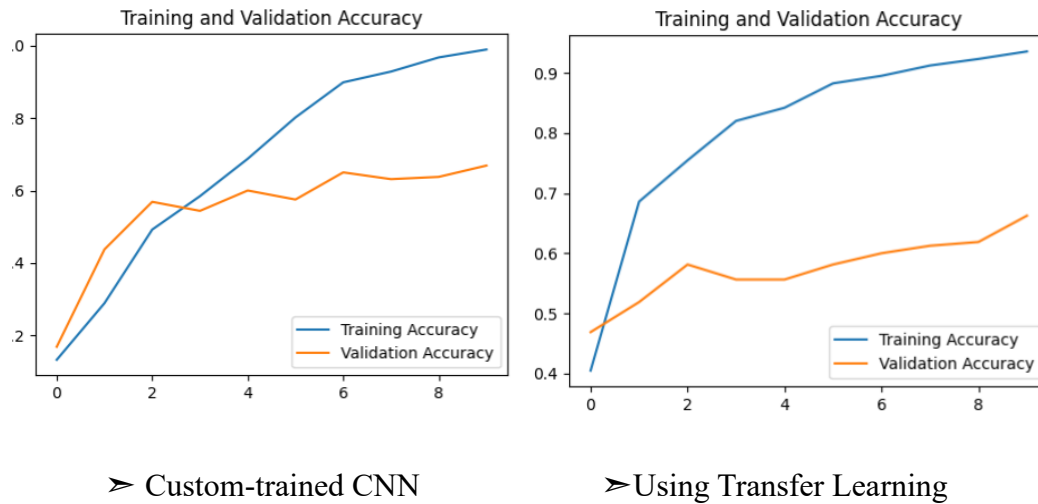
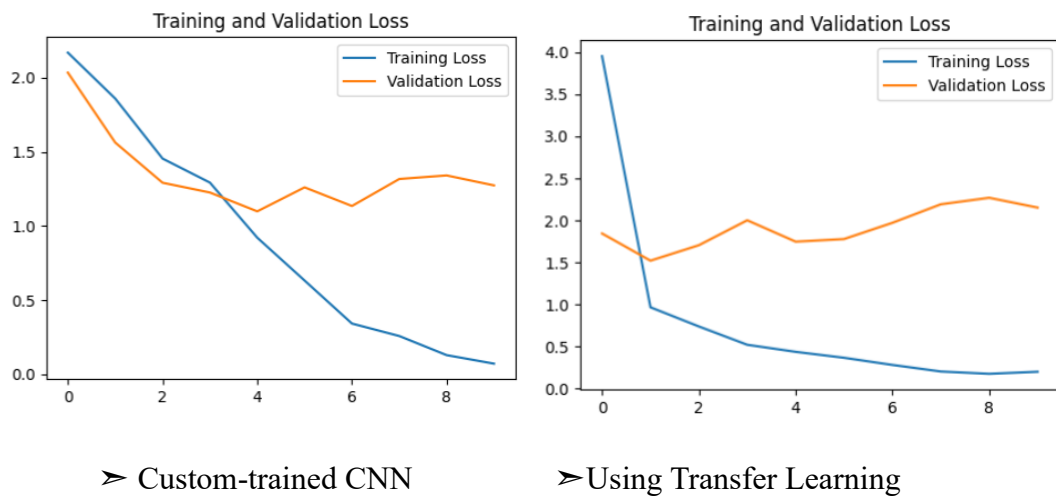


Figure 2: Training and Validation loss



Initial Training / Validation Accuracy

Custom-trained CNN	Using Transfer Learning
≈ 10% ~ 15%	≈ 40% ~ 45%

Underlying Mechanism

Random Weight Initialization	Loaded with weights='imagenet', only the final classification layers (Dense layers) were trained, while the earlier convolutional layers remained frozen.
------------------------------	---

In the 8-class classification task of this project, the baseline accuracy for random guessing is 12.5%. The custom-trained CNN model achieved an initial accuracy of approximately 14%, which essentially equates to random guessing. However, after introducing transfer learning, the model demonstrated an accuracy of over 40% right from the initial stage (Epoch 0), reaching more than three times the random guess baseline. This demonstrates the value of the ImageNet weights provided by ResNet50V2, which create a distinct advantage in feature extraction and effectively overcome the high initial learning costs associated with training from scratch.

Final Validation Loss

Custom-trained CNN	Using Transfer Learning
≈ 1.3	≈ 2.15

Due to the limited amount of information acquired, the custom-trained model exhibits extremely low confidence, yielding low probability scores regardless of whether its predictions are correct or incorrect. Conversely, the *ResNet* model utilizing transfer learning possesses exceptionally strong feature extraction capabilities, producing highly distinct features. This causes the appended classification layer (Dense Layer) to become "overconfident" when making predictions. Consequently, when the model makes a mistake, it does so with very high confidence. Under the calculation mechanism of *Categorical Crossentropy*, this highly confident incorrect prediction results in an extremely severe loss penalty.

Problem

Both approaches suffered from overfitting.

In both approaches, the training accuracy surged rapidly, **approaching nearly 100%**, while **the validation accuracy plateaued at around 65%**. Additionally, the validation loss for both methods initially showed a slight decrease but then reversed

and spiked upward. This indicates that our insufficient dataset size caused the model to merely memorize the training images. Even with the use of transfer learning, because the model capacity exceeded the diversity of the data, both methods ultimately led to overfitting.

Final Validation Accuracy

Custom-trained CNN	Using Transfer Learning
66.87%	66.25%

Insufficient data, high feature overlap, mismatched distributions between the training and validation sets, and overfitting collectively caused the final validation accuracy for both approaches to plateau **at approximately 66%**.

Furthermore, the validation accuracy of the transfer learning model using *ResNet50V2* was marginally lower than that of the custom-built shallow CNN. This phenomenon is likely primarily driven by a "Domain Gap." *ResNet's* pre-trained weights are highly focused on *ImageNet's* object-centric features, whereas the library classification task is inherently scene-centric. Under the strategy of freezing the convolutional layers, *ResNet* was unable to effectively adapt its receptive fields to capture the textures of architectural spaces.

In summary, this comparative analysis demonstrates that while Transfer Learning (*ResNet50V2*) provides a significant initial advantage through its pre-trained feature extraction capabilities, it cannot bypass the limitations of a constrained dataset. Both the custom-trained CNN and the transfer learning model ultimately suffered from severe overfitting, plateauing at approximately 66% validation accuracy.

The custom model struggled with feature extraction from scratch, resulting in low-confidence predictions. Conversely, *ResNet50V2's* massive capacity, combined with a distinct "Domain Gap" between general object recognition and specific architectural scenes, led to overconfident misclassifications and a significantly inflated validation loss. Ultimately, this comparison highlights a core machine learning principle: when faced with inherent dataset flaws—such as limited size, high feature overlap, and imbalanced distribution—simply adopting a more complex, pre-trained model is

insufficient to overcome the fundamental lack of data richness.

Results

Summary

The theme of our project, "SEE-SEE-U," aims to reveal hidden location information in social media photos, using a CNN model as its technical core. By accurately extracting landmark features, the model connects everyday photos with real geographical locations. This effectively solves a real user pain point: no longer missing out on new places because creators hide locations or because users feel awkward asking unfamiliar followers.

We integrated this model into a LINE chatbot to create a highly convenient interface. Users simply upload a photo within a familiar app, and the system instantly provides a Google Maps navigation link. Additionally, a built-in 0.75 confidence threshold filters out blurry or irrelevant images, ensuring accuracy while maintaining a seamless "upload-and-navigate" experience. Overall, this project validates the model's technical value and proves the system's genuine practical utility in solving real-world problems.

Recommendation

Based on our experience from this project implementation, we propose the following specific optimization recommendations to address the system's current technical limitations and support its future development:

- **Filtering Irrelevant Photos:** During testing, we found the system cannot effectively identify non-building images. If a user uploads an irrelevant photo, the model is forced to predict one of the 8 library categories based on the highest probability, which can lead to misidentification despite the 0.75 threshold. To prevent incorrect navigation from random uploads, we recommend adding an "irrelevant object" category or integrating an object detection model to verify the presence of a building before classification.
- **Increasing Photo Diversity:** To improve the CNN model's stability, we recommend expanding our dataset by intentionally collecting campus photos taken under different weather conditions (such as rainy or cloudy days) and at various times of day (such as evening or night). This will effectively enhance

the model's adaptability to complex outdoor lighting environments.

- **Protecting User Privacy:** To ensure readiness for future applications, we recommend integrating an automatic blurring algorithm into the image pre-processing stage. Upon receiving a photo, the system should automatically detect and obscure any human faces or license plates, extracting features solely from the building structure. This will provide navigation convenience while maintaining strict compliance with privacy protection principles.

Future Outlook

Based on our current implementation results, we have also mapped out a future development blueprint for the system. Below are the directions for our future optimization and expansion:

- **Expanding to Everyday Locations:** According to our survey, the top two types of places users want to find are cafés (65.9%) and restaurants (58.5%). Therefore, we plan to train our model with images from these two categories in the future to make the system more practical for daily use.
- **Scaling to All of Taiwan and the World:** Although we are currently testing only with NCCU libraries, this design can gradually be applied to scenic spots and landmarks across Taiwan, and eventually the world, becoming a universal tool for landmark recognition and map navigation.

Contributors

The "SEE-SEE-U" project was successfully executed through the distinct, collaborative contributions of our team members:

- **Mica 黎芯妤 114zu1009(Project Conceptualization & Documentation)**
Formulated the original project vision and analyzed initial user needs from campus survey data. Authored the technical *Project Description* and *Getting Started* repository initialization documentation.
- **Helen 趙芯悅 114zu1025 (Repository Architecture & Workflow Management)**
Designed the directory blueprints and mapped entire end-to-end file dependencies matching the ResNet50V2 pipeline. Organized structural folder

modularity and systematically documented team workflows and administrative project responsibilities.

- **Gina 王紫君 114zu1001 (Core Machine Learning Engineering)**

Served as the lead algorithm developer. Built the training script, successfully implemented Transfer Learning using the **ResNet50V2** base network, and tuned the categorical training loops to achieve optimal multi-class accuracy.

- **Angle 汪宜家 114zu1015(Data Analytics & Visualization)**

Supervised data verification and compiled deep learning run metrics across execution stages. Engineered model performance visualization charts (Accuracy and Loss curves) and authored formal technical descriptions for optimization trends.

- **Chloe 楊恆慈 114zu1058 (Data Interpretation & Evaluation Strategy)**

Analyzed raw evaluation scores against practical campus environment demands. Interpreted the real-world utility of the 0.75 confidence threshold and drafted final scalability blueprints for prospective production upgrades.

Acknowledgments

The successful completion of this project would not have been possible without the guidance and support of many individuals. First and foremost, we would like to express our sincere gratitude to the professor of the *Introduction to Artificial Intelligence* course at National Chengchi University for providing professional insights and inspiration regarding our model architecture design.

Furthermore, we are grateful to the course teaching assistants for their invaluable technical guidance and patience in helping us resolve Google Colab environment configurations and debugging programming errors.

Lastly, we extend our appreciation to all our team members for their seamless collaboration in web scraping, data collection, and front-end development. It is through the joint dedication of every member that the "SEE-SEE-U" system was successfully developed, achieving both technical implementation and practical utility for our campus community.

References

ResNet50V2

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Identity mappings in deep residual networks. In *European conference on computer vision* (pp. 630-645). Springer, Cham.

Adam

Kingma, D. P., & Ba, J. (2014). Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*.

TensorFlow / Keras

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., ... & Zheng, X. (2015). *TensorFlow: Large-scale machine learning on heterogeneous systems*. Software available from tensorflow.org.

Bing Image Downloader

Khan, G. (2020). *bing-image-downloader: Python library to download imagery from Bing* (Version 1.1.2) [Computer software]. GitHub.
<https://github.com/ostrolucky/Bulk-Bing-Image-Downloader>

Google Maps

Google. (n.d.). *Google Maps Platform documentation*. Google Developer. Retrieved June 30, 2026, from <https://developers.google.com/maps>